Chapter 11B: Calculator

Welcome back to the second lesson in our calculator application. In this lesson we are going to learn how to write event-driven code, i.e. code which executes in response to user-initiated actions. For example, the primary event associated with a button object is the single click. When the user clicks on a button something should happen. We are going to learn how to make that click response.

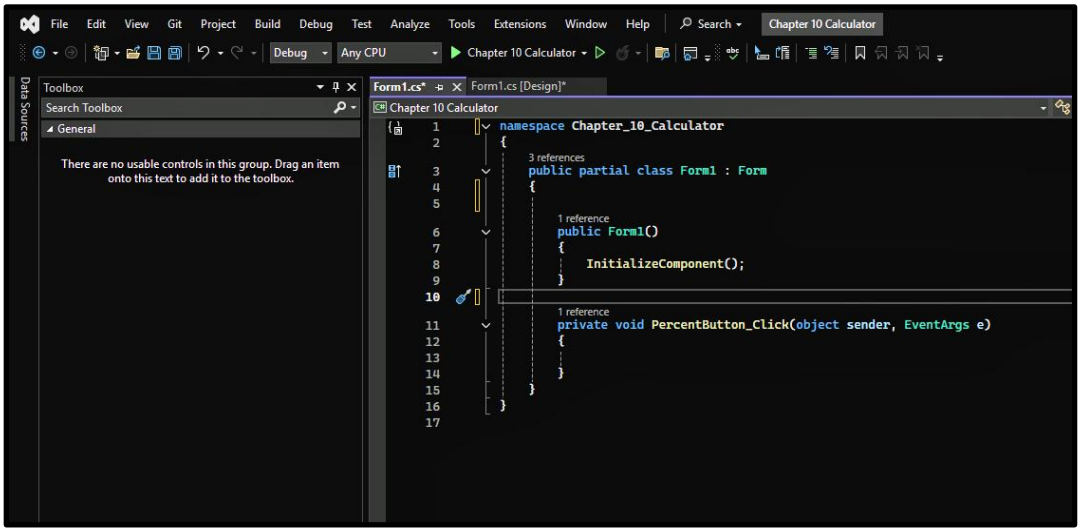
Okay, let's jump to the inside and write some event-driven code.

We begin of course by opening back up our calculator application. Let's get started writing some event-handler code.

If you already have a Form1.cs tab without the word Design in square brackets simply click that guy now to jump over to your code behind file. If you don't have that tab, go ahead and double click on any of the buttons on your form and the file will automatically be created for you.

I want you to check out this code so we can talk about why each element is here and what it does. (Figure 11B.1) First, check out the namespace declaration up at the top of the code. Mine inaccurately reads "namespace Chapter_10_Calculator" because I used this example once before in an earlier course where it was chapter 10. Yours should read "namespace Chapter_11_Calculator". Sorry for that confusion, my bad. I should have started from scratch instead.

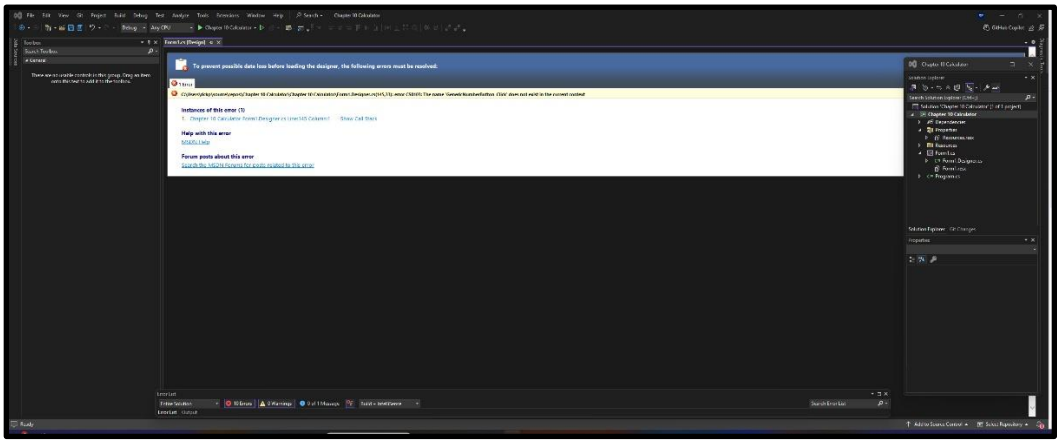
Figure 11B.1: Default Code Behind



Next comes the Form1 class inheriting from the base Form class. That syntax should be familiar. The public partial syntax simply means that this Form1 object is available to other routines running within the same solution, i.e. it is public. The partial indicates that the current class definition may be split across multiple source files within the current solution. The partial keyword tells the compiler to bring all those partial definitions together when actually defining the object in memory.

The public Form1() InitializeComponent() was written there by Visual Studio. The InitializeComponent routine contains all the code associated with painting the objects onto the form. Do not mess with that block of code or your entire project falls apart. If you mess up and get the screen shown here (Figure 11B.2) do not panic, your project can still be salvaged.

Figure 11B.2: Error Exists Screen



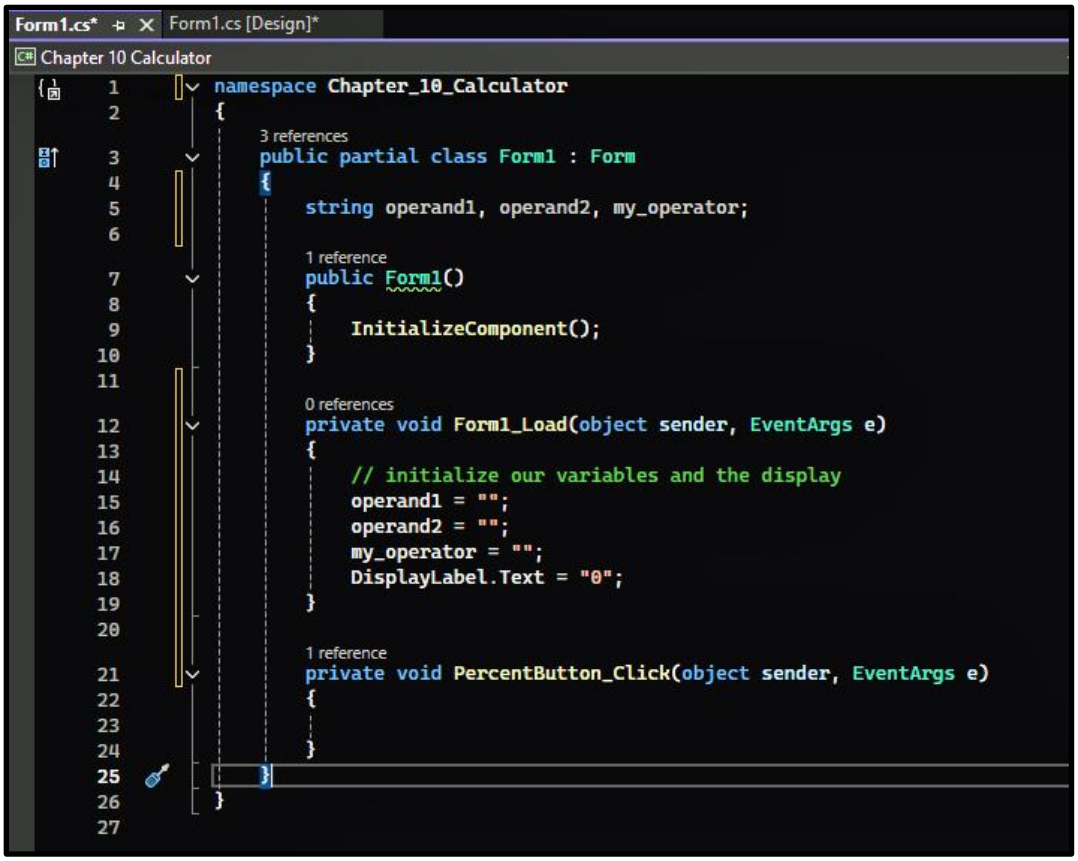
This error screen is telling me that at line 145 of my InitializeComponents file I am referencing a routine named GenericNumberButton_Click routine which does not exist. Apparently, I had previously linked one or more objects on my Form to that routine and now it does not exist making my project unable to run until the issue is resolved.

Okay, easy enough, just click on that link to open the file and remove all (emphasis all) erroneous references to the nonexistent routine. What is the lesson here? Never delete routines from your code behind file. If you don't need them anymore simply remove all the code from within that routine. The references remain intact, but nothing happens if that routine is called.

Finally, we see a private void PercentButton_Click(object sender, EventArgs e) routine. This was put there by Visual Studio when I double clicked the PercentButton. The default event for all buttons is the single click event, so when I double clicked the PercentButton Visual Studio guessed that I wanted to write a single click event handler for the PercentButton. The object sender parameter represents the object that raised the event, in our case the PercentButton. To reference properties of that object you simply cast it to its class type. We will see a couple examples of that cast later in this chapter. As the class name implies, the EventArgs class contains details about the event that raised the current event call.

Now let's update our code to look like that shown here. (Figure 11B.3) As you can see, I have added string variables for my two operands and the operator symbol. I have also added a Form1_Load event handler. The default event for a Form is its load event. Any code written in a Form's load event handler is executed immediately before the Form is shown to the user. In our case we are initializing our string variables and the display label.

Figure 11B.3: Form1_Load



I next want to write the code behind my ClearButton and ClearEntryButton. The first cut on my code is shown here. (Figure 11B.4) When I had a chance to refine things a little bit I came up with the code shown here. (Figure 11B.5) Okay, what did we learn here with my code refinement process? Never write the same logic in multiple locations within a solution. It means more maintenance and a greater opportunity for errors. The rule is this, “if you have multiple copies of the same logic within your solution, you will always discover their inconsistencies while your boss is standing behind you looking over your shoulder.” Most definitely not cool. Oh, and remember, do not delete the old routines until you have linked the Clear and ClearRntry Buttons to the new generic routine. If you forget you will get that inconsistency error screen shown earlier and you will have to go clear out all the bad references.

Figure 11B.4: ClearButton_Click and ClearEntryButton_Click

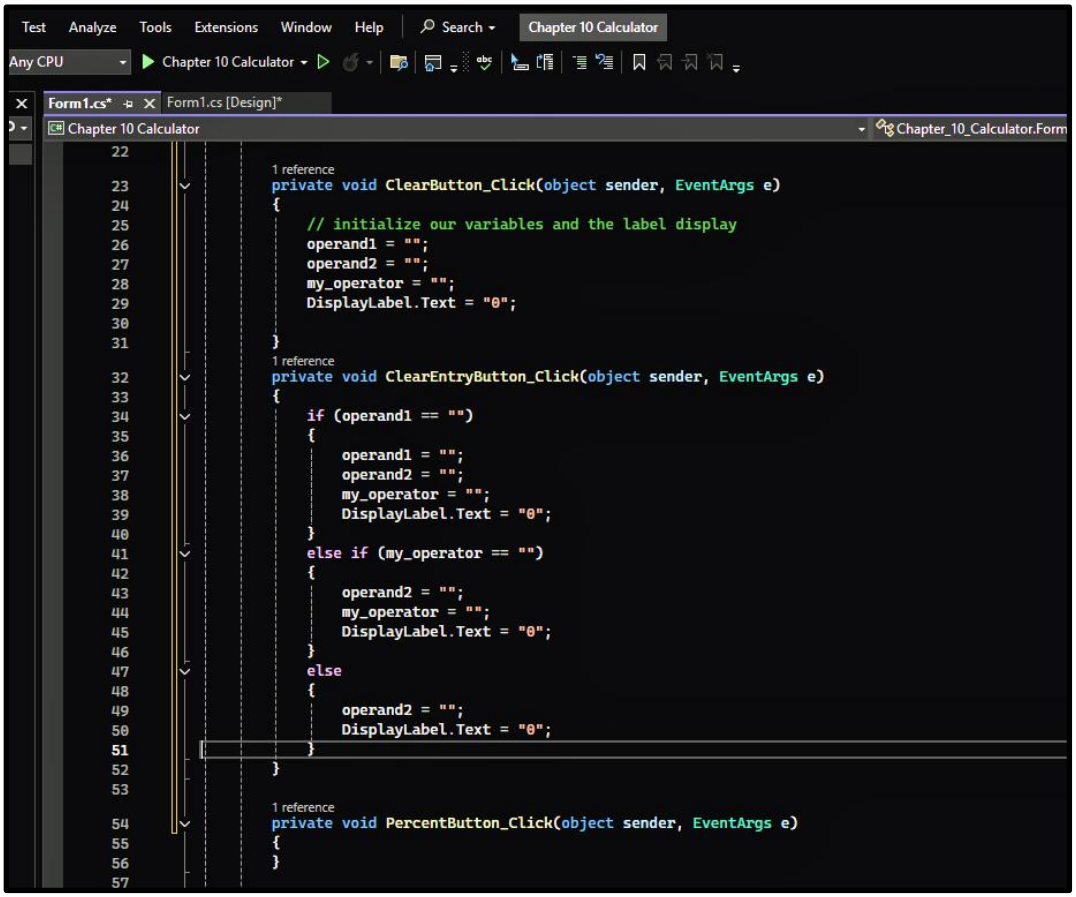
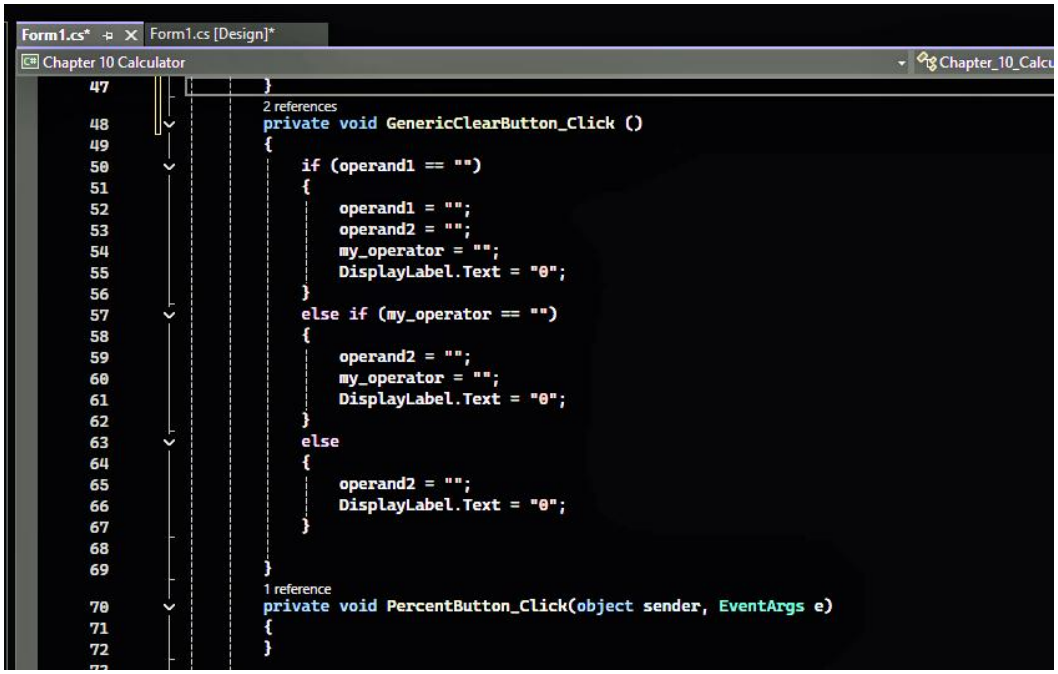
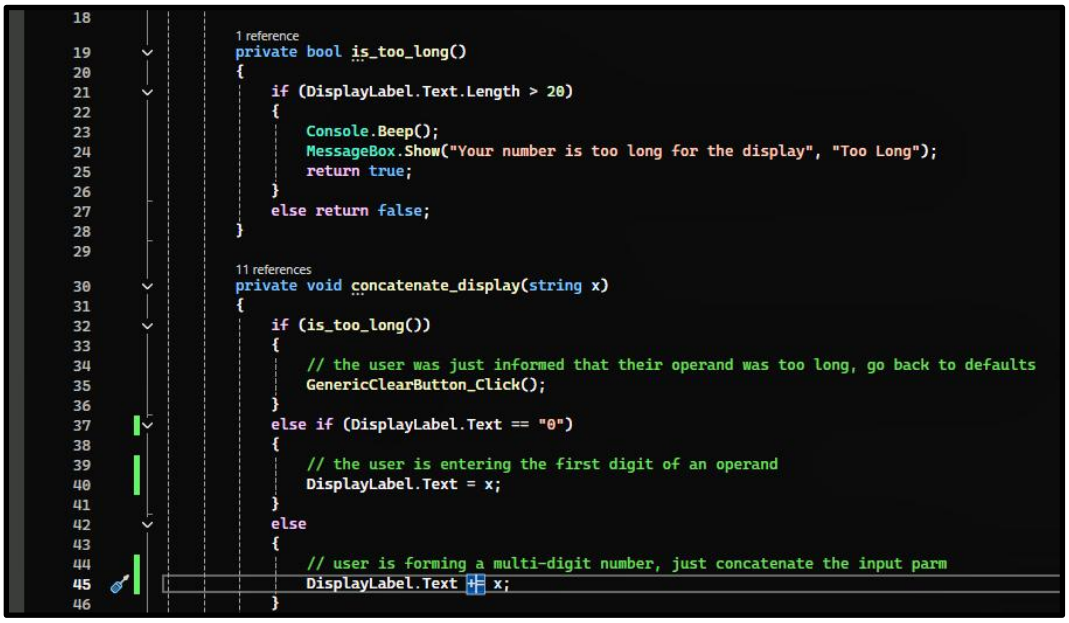


Figure 11B: 5 GenericClearButton_Click



I next wrote the two routines shown here. (Figure 11B.6) The is_too_long routine checks the length of the display text and sends a beep and message to the user if the routine resolves to true. The concatenate_display routine accepts a single string parameter and either resets all our variables when the display label was too long, concatenates that string to the current display, or sets the display label to the input string. We of course will be linking the concatenate routine to each of our digit buttons.

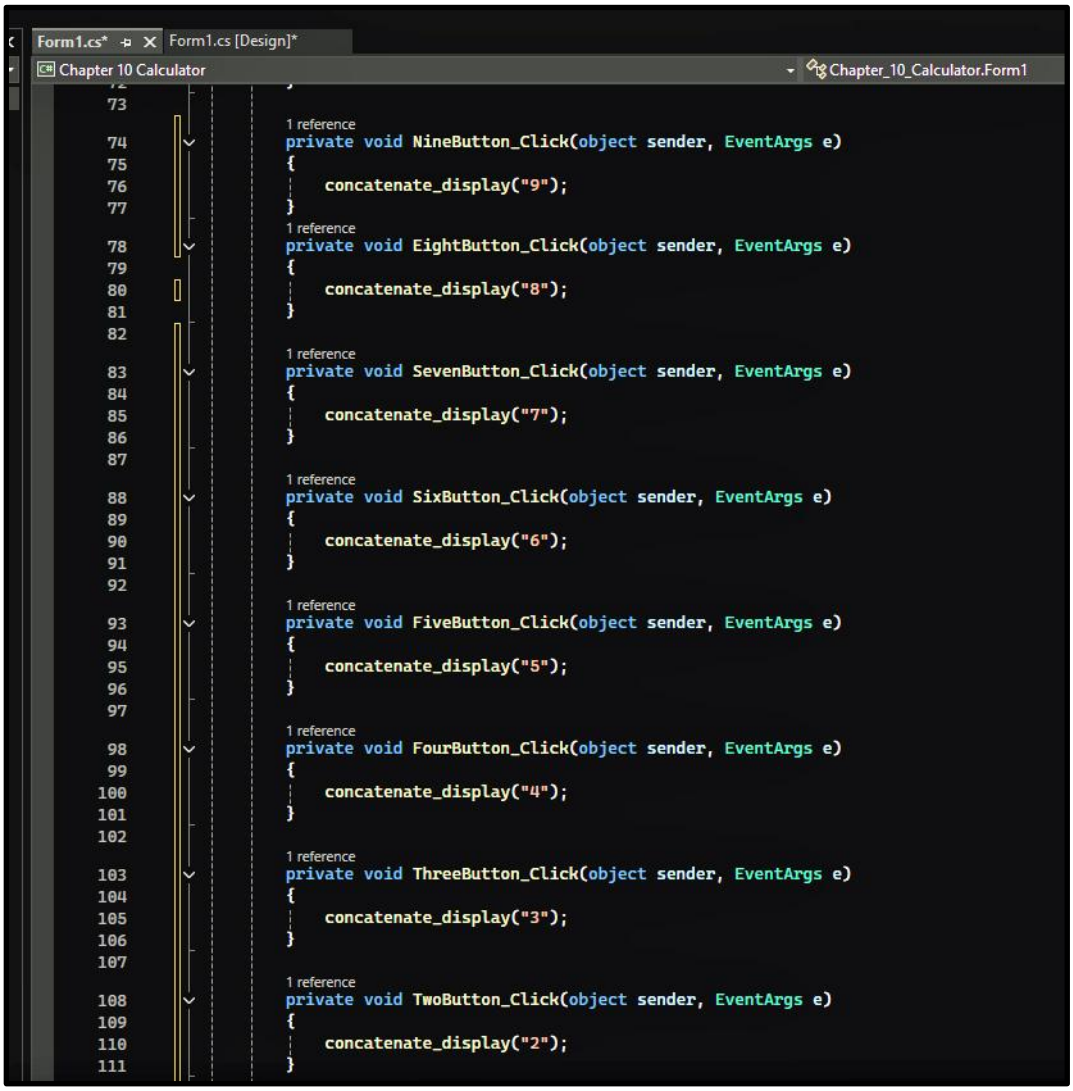
Figure 11B.6: is_too_long and concatenate_display



```
18
                1 reference
19              private bool is_too_long()
20              {
21                  if (DisplayLabel.Text.Length > 20)
22                  {
23                      Console.Beep();
24                      MessageBox.Show("Your number is too long for the display", "Too Long");
25                      return true;
26                  }
27                  else return false;
28              }
29
                11 references
30              private void concatenate_display(string x)
31              {
32                  if (is_too_long())
33                  {
34                      // the user was just informed that their operand was too long, go back to defaults
35                      GenericClearButton_Click();
36                  }
37                  else if (DisplayLabel.Text == "0")
38                  {
39                      // the user is entering the first digit of an operand
40                      DisplayLabel.Text = x;
41                  }
42                  else
43                  {
44                      // user is forming a multi-digit number, just concatenate the input parm
45                      DisplayLabel.Text += x;
46                  }
```

I next linked each of our digit buttons to the concatenate_display routine by double clicking on them and then updating the routine created by Visual Studio. The result shown here works but lacks elegance. (Figure 11B.7) No programmer worth their salt would be happy with this solution.

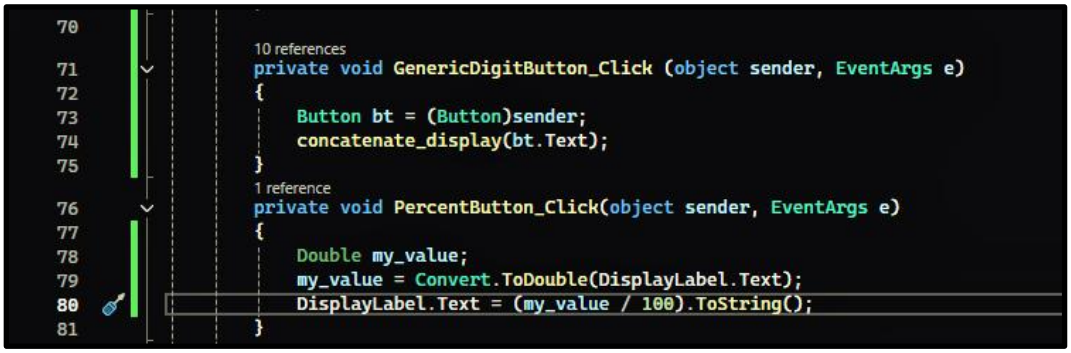
Figure 11B.7: Inelegant Digit Button Click Routines



The solution shown here is much more elegant and far easier to maintain. (Figure 11B.8) Remember that sender parameter describes the object which activated the current event. By casting that guy to a Button type we can then use that button's text property as the input parameter to our concatenate routine. Now that is slick. Remember however not to delete the old digit click routines until you have gone back to the design page and changed the routine associated with each of the digit buttons.

By the way, while I was in this part of my code I went ahead and completed my PercentButton_Click routine. According to Google, the source of all knowledge, on a simple 4-function calculator the percent button should take the currently displayed value and divide it by 100. Mine works, interestingly enough my Windows 11 calculator does not. It returns a value of zero regardless of the previous value of the display. Again, nice software QA Microsoft.

guysFigure 11B.8: GenericDigitButton_Click and PercentButton_Click

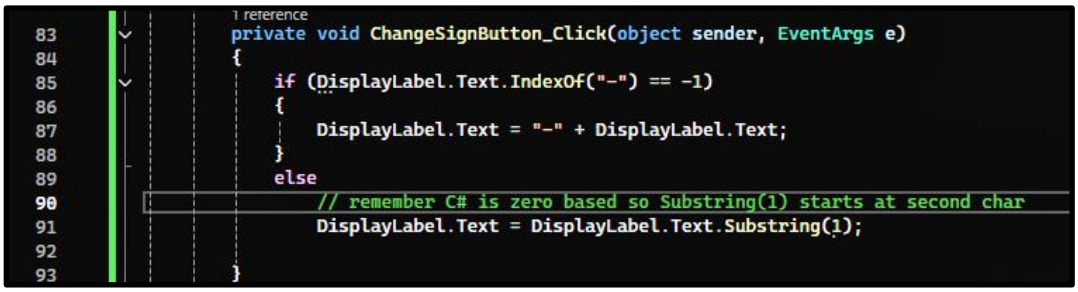


```
70
                10 references
71              private void GenericDigitButton_Click (object sender, EventArgs e)
72              {
73                  Button bt = (Button)sender;
74                  concatenate_display(bt.Text);
75              }
                1 reference
76              private void PercentButton_Click(object sender, EventArgs e)
77              {
78                  Double my_value;
79                  my_value = Convert.ToDouble(DisplayLabel.Text);
80                  DisplayLabel.Text = (my_value / 100).ToString();
81              }
```

What are we learning about good coding technique gang? Keep it generic and simple such that when someone else inherits your code it is immediately apparent to them how it works and why it was done the way it was.

My ChangeSignButton_Click routine is equally straight forward. Remember early on when we learned about the IndexOf and Substring functions? Well, now you get to see them in action. (Figure 11B.9)

Figure 11B.9: ChangeSignButton_Click



```
                1 reference
83              private void ChangeSignButton_Click(object sender, EventArgs e)
84              {
85                  if (DisplayLabel.Text.IndexOf("-") == -1)
86                  {
87                      DisplayLabel.Text = "-" + DisplayLabel.Text;
88                  }
89                  else
90                      // remember C# is zero based so Substring(1) starts at second char
91                      DisplayLabel.Text = DisplayLabel.Text.Substring(1);
92
93              }
```

So, what routine do we associate with the decimal point button? Of course, that is just another concatenate situation so we associate the GenericDigitButton_Click routine. By the way, I am testing my form after each of these modifications. If everything is working properly, I do a save. If something went south, I close without saving and go back to my last properly functioning solution.

I next brought up my Windows calculator again and entered a value of 15. When I hit the plus sign the 15 stays displayed until I hit the next digit button. I then entered a value of 60 and hit the equals sign button. The display read 75 as expected. But what happens if I enter a value of 15, hit the plus sign, enter a value of 60 and then hit the plus sign again? My calculator now informs me that the new value for operand1 is 75 and there is no value for operand2. Okay, this is going to require some modification to our concatenate routine.

I began by adding a boolean type variable named operand1_set to the top of my code block as shown here. I am going to use that guy to flag if and when operand1 has a value.

```
3 references
public partial class Form1 : Form
{
    string operand1, operand2, my_operator;
    bool operand1_set;
```

I next had to modify my GenericClearButton_Click routine to set that guy to false when operand1 had no value.

```
2 references
57  private void GenericClearButton_Click()
58  {
59      if (operand1 == "")
60      {
61          operand1 = "";
62          operand2 = "";
63          my_operator = "";
64          DisplayLabel.Text = "0";
65          operand1_set = false;
66      }
67      else if (my_operator == "")
68      {
69          operand2 = "";
70          my_operator = "";
71          DisplayLabel.Text = "0";
72      }
73      else
74      {
75          operand2 = "";
76          DisplayLabel.Text = "0";
77      }
78
79  }
80
```

I next had to update my concatenate_display routine to accommodate my operand1_set flag. Oh yes, programmers often refer to this type of bool variable as a flag as it serves to flag if some condition exists. In our case, whether the user has already set operand1 to some value.

```
1 reference
private void concatenate_display(string x)
{
    if (is_too_long())
    {
        // the user was just informed that their operand was too long, go back to defaults
        GenericClearButton_Click();
    }
    else if (DisplayLabel.Text == "0" || operand1_set == true)
    {
        if (DisplayLabel.Text == "0" || DisplayLabel.Text == operand1)
        {
            // the user is entering the first digit of an operand
            DisplayLabel.Text = x;
        }
        else
        {
            // the user is entering additional digits to operand1
            DisplayLabel.Text += x;
        }
    }
    else
    {
        // user is forming a multi-digit number, just concatenate the input parm
        DisplayLabel.Text += x;
    }
}
```

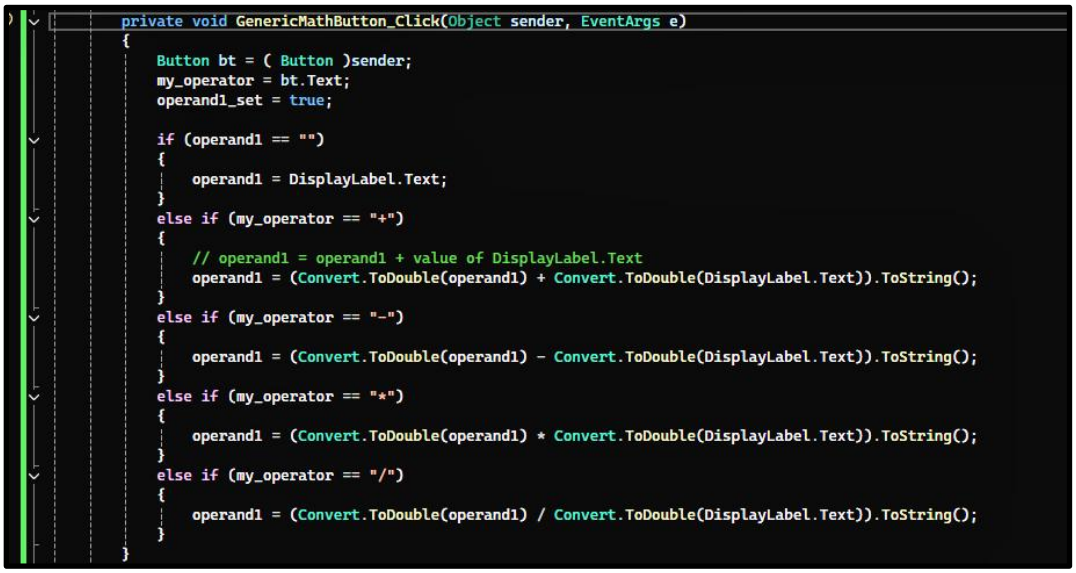
With those modifications behind me I was finally ready to write my AddButton_Click routine. Please note how I had to cast operand1 and the DisplayLabel text to double before adding them and then had to cast the result back to a string before assigning the new value back to operand1. (Figure 11B.10)

Figure 11B.10: AddButton_Click

```
1 reference
private void AddButton_Click(object sender, EventArgs e)
{
    operand1_set = true;
    if (operand1 == "")
    {
        operand1 = DisplayLabel.Text;
    }
    else
    {
        // operand1 = operand1 + value of DisplayLabel.Text
        operand1 = (Convert.ToDouble(operand1) + Convert.ToDouble(DisplayLabel.Text)).ToString();
    }
}
```

Okay, at this point I went back to my Windows calculator and found that the minus, multiply and divide keys all worked just like my plus sign. Time to change my AddButton_Click routine to a more generic version. (Figure 11B.11) Oh and don't forget to change that linkage before you delete the old add routine.

Figure 11B.11: GenericMathButton_Click

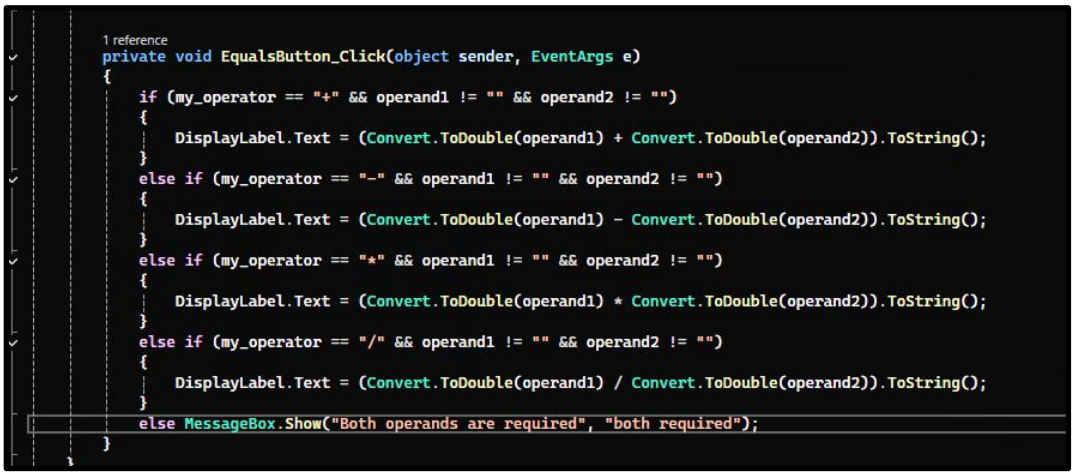


```
private void GenericMathButton_Click(Object sender, EventArgs e)
{
    Button bt = ( Button )sender;
    my_operator = bt.Text;
    operand1_set = true;

    if (operand1 == "")
    {
        operand1 = DisplayLabel.Text;
    }
    else if (my_operator == "+")
    {
        // operand1 = operand1 + value of DisplayLabel.Text
        operand1 = (Convert.ToDouble(operand1) + Convert.ToDouble(DisplayLabel.Text)).ToString();
    }
    else if (my_operator == "-")
    {
        operand1 = (Convert.ToDouble(operand1) - Convert.ToDouble(DisplayLabel.Text)).ToString();
    }
    else if (my_operator == "*")
    {
        operand1 = (Convert.ToDouble(operand1) * Convert.ToDouble(DisplayLabel.Text)).ToString();
    }
    else if (my_operator == "/")
    {
        operand1 = (Convert.ToDouble(operand1) / Convert.ToDouble(DisplayLabel.Text)).ToString();
    }
}
```

Well, we only have one button left to handle so let's take a look at our EqualsButton_Click routine. (Figure 11B.12) Pretty straight forward stuff.

Figure 11B.12: EqualsButton_Click

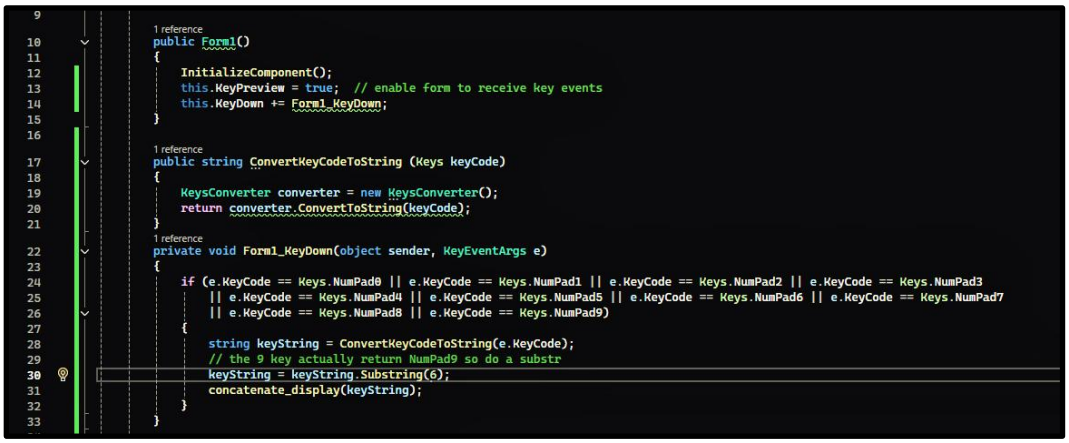


```
1 reference
private void EqualsButton_Click(object sender, EventArgs e)
{
    if (my_operator == "+" && operand1 != "" && operand2 != "")
    {
        DisplayLabel.Text = (Convert.ToDouble(operand1) + Convert.ToDouble(operand2)).ToString();
    }
    else if (my_operator == "-" && operand1 != "" && operand2 != "")
    {
        DisplayLabel.Text = (Convert.ToDouble(operand1) - Convert.ToDouble(operand2)).ToString();
    }
    else if (my_operator == "*" && operand1 != "" && operand2 != "")
    {
        DisplayLabel.Text = (Convert.ToDouble(operand1) * Convert.ToDouble(operand2)).ToString();
    }
    else if (my_operator == "/" && operand1 != "" && operand2 != "")
    {
        DisplayLabel.Text = (Convert.ToDouble(operand1) / Convert.ToDouble(operand2)).ToString();
    }
    else MessageBox.Show("Both operands are required", "both required");
}
```

Let's do one more thing before we wrap this example up. My Windows calculator lets me use my keyboard to make entries. Let's give our calculator that same capability.

In my Form1_KeyDown routine I only implement the digit keys on the number pad. (Figure 11B.13) The same strategy could have also been used for the upper row digit keys and our math operators but this lesson is already getting far too long. If you want an interesting exercise, go ahead and implement those yourself. Hint, you will need to update our current GenericMathButton_Click routine to put the code which assigns the my_operator variable off of the button click inside a conditional. Oh, and did you notice I also had to update my Form1 constructor routine to add the Form1_Keydown method.

Figure 11B.13: Form1_KeyDown



```
9
                1 reference
10              public Form1()
11              {
12                  InitializeComponent();
13                  this.KeyPreview = true;  // enable form to receive key events
14                  this.KeyDown += Form1_KeyDown;
15              }
16
                1 reference
17              public string ConvertKeyCodeToString (Keys keyCode)
18              {
19                  KeysConverter converter = new KeysConverter();
20                  return converter.ConvertToString(keyCode);
21              }
                1 reference
22              private void Form1_KeyDown(object sender, KeyEventArgs e)
23              {
24                  if (e.KeyCode == Keys.NumPad0 || e.KeyCode == Keys.NumPad1 || e.KeyCode == Keys.NumPad2 || e.KeyCode == Keys.NumPad3
25                      || e.KeyCode == Keys.NumPad4 || e.KeyCode == Keys.NumPad5 || e.KeyCode == Keys.NumPad6 || e.KeyCode == Keys.NumPad7
26                      || e.KeyCode == Keys.NumPad8 || e.KeyCode == Keys.NumPad9)
27                  {
28                      string keyString = ConvertKeyCodeToString(e.KeyCode);
29                      // the 9 key actually return NumPad9 so do a substr
30                      keyString = keyString.Substring(6);
31                      concatenate_display(keyString);
32                  }
33              }
```

As I just mentioned, this lesson is getting very long so let's jump to the outside and do our review.

So, some pretty interesting code behind our calculator buttons wasn't there. What did we learn?

- How to create and link an event handler to an object on our form.
- How to convert multiple repetitive routines into a single generic routine associated with multiple objects on the window.
- How to capture keystrokes.
- How software kind of evolves over time. The first version of a routine is very seldom the final version as changes are often required as new routines are created.

Okay, in our next example we build a matching game, should be interesting.